Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

EDGE-AI FRAMEWORK FOR REAL TIME SURFACE ANOMALY DETECTION

Submitted by

HARIHARAN S (234011101544)

In partial fulfillment of the Requirements for the award of the degree

of

BACHELOR

OF

COMPUTER APPLICATIONS

APRIL 2026

DECLARATION FORMAT

I _____ hereby declare that the Project Report entitled "_____

_____" is done by me under the guidance of Dr./Prof./Mr./Ms _____(Internal) is submitted in partial fulfillment of the requirements for the award of the degree in BACHELOR OF COMPUTER APPLICATIONS.

DATE:

PLACE: **SIGNATURE OF THE CANDIDATE**

Dr. M.G.R.
EDUCATIONAL AND RESEARCH INSTITUTE
DEEMED TO BE UNIVERSITY
University with Graded Autonomy Status
(An ISO 21001 : 2018 Certified Institution)
Periyar E.V.R. High Road, Maduravoyal, Chennai-95. Tamilnadu, India.

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of Mr./Ms._____ _____ Reg. No_____who carried out the project entitled "_____

_____" under our supervision from December 2025 to March 2026.

Internal Guide **Head of the Department**

Submitted for Viva Voce Examination held on_____

Internal Examiner **External Examiner**

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
	LIST OF FIGURES	2
	LIST OF TABLES	3
1	**INTRODUCTION**	4
	1.1 About	4
	1.2 Problem statement	5
	1.3 Objectives of the project	5
	1.4 Scope of the project	5
	1.5 Overview of the proposed system	6
2	**EXISTING SYSTEM**	7
	2.1 Introduction	7
	2.2 Manual visual inspection	7
	2.3 Classical image processing techniques	8
	2.4 Cloud based AI systems	8
	2.5 High end industrial inspection systems	9
	2.6 AI based detection systems	9
	2.7 Summary of limitations	10
	2.8 Need for proposed system	10
3	**PROPOSED SYSTEM**	12
	3.1 Introduction	12
	3.2 Overview of the proposed system	12
	3.3 Key features of the proposed system	12
	3.3.1 Edge based processing	12
	3.3.2 Distributed architecture(Major design improvement)	13

3.3.3 Real time detection 13
3.3.4 Wireless streaming 13
3.3.5 Lightweight deep learning model 13
3.3.6 Portability and cost effectiveness 13
3.4 Dataset preparation and enhancement 14
3.4.1 Dataset composition 14
3.4.2 Dataset cleaning 14
3.4.3 Annotation verification 14
3.4.4 Impact of dataset improvements 14
3.5 Training optimization 14
3.6 Model optimization and performance enhancement 15
3.6.1 Inference optimization 15
3.6.2 Model conversion 15
3.7 Hardware optimization 15
3.8 System architecture 15
3.9 Performance evaluation 16
3.10 Summary 17

4 LITERATURE REVIEW 18
4.1 Introduction 18
4.2 Traditional methods 18
4.3 Deep learning-based approaches 18
4.4 Edge AI in computer vision 18
4.5 Research gap 19
4.6 Contribution of proposed work 19

5 METHODOLOGY 20
5.1 Introduction 20
5.2 Learning approach 20
5.3 Data representation 20

5.4 Convolutional neural network (CNN) concept 20
5.4.1 Convolutional operation 20
5.4.2 Activation function (ReLU) 20
5.4.3 Pooling layer 20
5.5 YOLOV8 detection mechanism 21
5.6 Loss function 21
5.7 Training Strategy 21
5.8 Evaluation metrices 22
5.9 Inference logic 22
5.10 Summary 22

6 HARDWARE AND SOFTWARE REQUIREMENTS 23
6.1 Hardware requirements 23
6.2 Software requirements 23
6.3 Functional requirements 23
6.4 Non functional requirements 23

7 SYSTEM ARCHITECTURE 24
7.1 Introduction 24
7.2 Overview of the architecture 24
7.3 System components 24
7.3.1 Camera module (Input acquisition) 24
7.3.2 Edge device – Raspberry Pi 4 25
7.3.3 Processing system (Pc/Laptop) 25
7.3.4 Detection model (YOLOV8) 25
7.3.5 Streaming module (Flask-based MJPEG streaming) 25
7.3.6 Display and visualization module 25
7.4 Working of the system 26
7.5 Distributed architecture design 26
7.6 Data flow in the system 27
7.7 Advantages of the architecture 27
7.8 Summary 28

8	**SYSTEM MODULES**	29
	8.1 Introduction	29
	8.2 Modules overview	29
	8.3 Image acquisition module	29
	8.4 Preprocessing module	29
	8.5 Detection module	29
	8.6 Decision module	30
	8.7 Streaming module	30
	8.8 Summary	30
9	**DATA FLOW DIAGRAM(DFD)**	31
	9.1 Introduction	31
	9.2 Level 0(context diagram)	31
	9.3 Level 1 DFD	31
	9.4 Level 2 DFD	32
10	**UML DIAGRAMS**	33
	10.1 Introduction	33
	10.2 Use case diagram	34
	10.3 Sequence diagram	34
	10.4 Class diagram	35
	10.5 Summary	35
11	**IMPLEMENTATION**	36
	11.1 Introduction	36
	11.2 Implementation overview	36
	11.3 Dataset collection and preparation	36
	11.3.1 Dataset composition	36
	11.3.2 Annotation verification	37
	11.3.4 Dataset structuring	37
	11.4 Model training	37
	11.4.1 Training environment	37
	11.4.2 Training configurations	37

11.4.3 Data augmentations 37
11.4.4 Training optimization 38
11.5 Training parameters 38
11.6 Model optimization 39
11.6.1 Inference optimization 39
11.6.2 Model conversion 39
11.7 System integration 40
11.7.1 Hardware setup 40
11.7.2 Software stack 40
11.7.3 Distributed processing implementation 40
11.8 Real – time detection system 40
11.9 Streaming implementation 41
11.10 Hardware optimization 41
11.11 Visual and interface implementation 41
11.12 Camera calibration and image processing 41
11.13 Performance evaluation 41
11.14 Challenges faced 41
11.15 Results 42
11.16 Summary 43

12 RESULTS AND DISCUSSION 45
12.1 Introduction 45
12.2 Training results analysis 45
12.3 Precision recall analysis 45
12.4 Precision analysis 45
12.5 Recall analysis 45
12.6 F1 score analysis 48
12.7 Confusion matrix analysis ‘49
12.8 Performance metrices 50
12.9 Real time performance analysis 51
12.10 Overall discussion 51
12.11 Summary 51

13 FEASIBILITY STUDY 52
13.1 Introduction 52
13.2 Technical feasibility 52
13.3 Economic feasibility 52
13.4 Operational feasibility 52
13.5 Feasibility summary 52

14 APPLICATIONS 53
14.1 Introduction 53
14.2 Infrastructure monitoring 53
14.3 Industrial inspection 53
14.4 Smart cities and public safety 53
14.5 Remote and portable inspection 54
14.6 Transportation and road safety 54
14.7 Future applications 54
14.8 Summary 54

15 CHALLENGES AND FUTURE WORK 55
15.1 Challenges faces 55
15.1.1 Dataset challenges 55
15.1.2 Model training challenges 55
15.1.3 Real time performance challenges 55
15.1.4 Environmental challenges 55
15.2 Limitations 55
15.3 Future improvements 55

16 CONCLUSION 57
16.1 Summary of the project 57
16.2 Key achievements 57
16.3 Significance of the work 57
16.4 Future scope 57
16.5 Final remarks 57

REFERENCES 58
APPENDICES 59

ABSTRACT

This project presents an Edge-AI framework for real-time surface anomaly detection using deep learning and computer vision techniques. The system is designed to identify surface defects, particularly cracks, from live video streams with high accuracy and low latency. The proposed solution utilizes a lightweight YOLOv8n object detection model optimized for real-time performance. A distributed architecture is implemented, where a Raspberry Pi equipped with a camera module captures and streams video frames over a local network, while a more powerful processing system performs deep learning inference. This approach significantly reduces computational load on the edge device while maintaining efficient real-time detection.

The system incorporates dataset refinement techniques, including data augmentation, annotation verification, and the inclusion of negative samples to improve model robustness and reduce false positives. Additional optimizations such as input resolution tuning, confidence threshold adjustment, and model conversion to ONNX format further enhance performance.

The framework achieves high accuracy with a precision of approximately 0.96 and operates at real-time speeds of 15–20 frames per second. The system is portable, cost-effective, and independent of cloud infrastructure, making it suitable for deployment in remote and real-world environments.

This work demonstrates the potential of combining edge computing with deep learning to develop scalable and efficient automated inspection systems for infrastructure monitoring, industrial applications, and public safety.

LIST OF FIGURES

Figure No	Figures	Page no
7.1	System architecture	9
9.1	Level 0 data flow diagram (context diagram)	9
10.1	Use case diagram of the proposed system	16
10.2	Sequence diagram of the proposed system	18
10.3	Class diagram of the proposed system	19
10.15.1	Sample outputs and results	21
10.15.2	Input frame captured from camera	22
12.1	Training and validation metrics	22
12.2	Precision recall curve	24
12.3	Precision curve	24
12.4	Recall curve	25
12.5	F1 score curve	25
12.6	Confusion matrix	25

LIST OF TABLES

Table No	Table	Page no
2.1	Comparison of existing surface anomaly detection system	9
3.1	Comparison of Existing Systems and Proposed Edge-AI System	
11.1	Training configuration parameters	9
11.2	Performance metrics of YOLOV8n model	16

CHAPTER 1
INTRODUCTION

1.1 ABOUT

Surface inspection plays a critical role in maintaining the structural integrity and safety of infrastructure such as buildings, roads, and industrial equipment. Traditionally, inspection processes are carried out manually by human experts, which makes them time-consuming, subjective, and prone to errors due to fatigue and inconsistency. Early-stage anomalies, such as fine cracks, are often difficult to detect with the naked eye, leading to delayed maintenance and increased risk of structural failure.

With the rapid advancement of Artificial Intelligence (AI) and Computer Vision, automated inspection systems have emerged as a promising alternative to manual methods. Deep learning models, particularly object detection algorithms, have demonstrated high accuracy in identifying visual defects in images and videos. However, many existing solutions rely heavily on cloud-based processing, which introduces latency, requires constant internet connectivity, and limits real-time deployment in remote or field environments.

To address these challenges, this project proposes an **Edge-AI framework for real-time surface anomaly detection**. The system leverages a lightweight deep learning model, YOLOv8n, to perform efficient object detection. A camera module captures live video frames, which are processed to detect anomalies such as surface cracks. The system is designed to operate in real-time, ensuring immediate feedback and enabling faster decision-making.

A key feature of the proposed system is its **distributed architecture**, where the Raspberry Pi 4 is used for video capture and wireless streaming, while computationally intensive inference is performed on a more powerful system. This design significantly reduces the processing burden on the edge device while maintaining real-time performance.

The system also incorporates several optimizations, including dataset refinement, data augmentation, model tuning, and hardware improvements, resulting in a highly accurate and robust detection system. With performance metrics achieving high precision and recall, the framework demonstrates its effectiveness in real-world scenarios.

Overall, this project presents a **portable, cost-effective, and scalable solution** for automated surface inspection, eliminating the dependency on cloud infrastructure and enabling deployment in diverse environments.

1.2 PROBLEM STATEMENT

Manual inspection of surfaces is inefficient and unreliable due to human limitations such as fatigue, subjectivity, and inconsistency. Detecting early-stage anomalies like cracks is particularly challenging, especially in large-scale or inaccessible environments.

Existing automated systems often depend on expensive hardware or cloud-based processing, which introduces latency and requires continuous internet connectivity. These limitations make such systems unsuitable for real-time and field-based applications.

Therefore, there is a need for a **real-time, efficient, and portable anomaly detection system** that can operate independently of cloud infrastructure while maintaining high accuracy and reliability.

1.3 OBJECTIVES OF THE PROJECT

The main objectives of this project are:

- To design and develop a real-time surface anomaly detection system using deep learning
- To implement a lightweight object detection model (YOLOv8n) for efficient inference
- To perform anomaly detection on edge devices with minimal latency
- To enable wireless video streaming for real-time monitoring
- To improve detection accuracy through dataset enhancement and model optimization

- To develop a portable and scalable system suitable for real-world deployment

1.4 SCOPE OF THE PROJECT

The scope of this project focuses on detecting surface anomalies, specifically cracks, using computer vision and deep learning techniques. The system is designed to operate in real-time and can be extended to detect other types of defects such as corrosion, dents, or structural deformations.

The modular architecture allows flexibility in upgrading components such as the detection model, dataset, and hardware. The system is suitable for applications in infrastructure monitoring, industrial inspection, and maintenance automation.

1.5 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system consists of a camera module connected to a Raspberry Pi 4, which captures real-time video frames. These frames are streamed wirelessly over a local network using an MJPEG-based streaming system. The inference is performed using a YOLOv8n model trained on a combination of publicly available and custom-annotated datasets.

The model has been optimized through techniques such as data augmentation, hyperparameter tuning, and resolution adjustment to achieve a balance between speed and accuracy. Additional improvements, including the use of negative samples and dataset cleaning, have significantly reduced false positives and improved model reliability.

The system achieves real-time performance with an approximate frame rate of 15–20 FPS and demonstrates high detection accuracy, making it suitable for practical deployment.

CHAPTER 2
EXSISTING SYSTEM

2.1 INTRODUCTION

Surface anomaly detection has been an important area of research in computer vision and industrial inspection. Over the years, various methods have been developed to identify defects such as cracks, corrosion, and deformations. These methods range from manual inspection techniques to advanced AI-based systems. However, each approach has its own limitations in terms of accuracy, scalability, cost, and real-time applicability.

This chapter discusses the commonly used existing systems for surface inspection and highlights their drawbacks, which motivate the need for the proposed Edge-AI-based solution.

2.2 MANUAL VISUAL INSPECTION

Manual inspection is the most traditional method used for detecting surface anomalies. In this approach, trained personnel visually examine structures and identify defects such as cracks or damages.

Advantages

- Does not require complex technology or equipment
- Flexible and adaptable to different inspection environments

Limitations

- Time-consuming and labor-intensive
- Highly subjective and dependent on human expertise
- Prone to errors due to fatigue and inconsistency
- Difficult to scale for large or inaccessible structures

Manual inspection is inefficient for continuous monitoring and often fails to detect fine or early-stage anomalies.

2.3 CLASSICAL IMAGE PROCESSING TECHNIQUES

Traditional computer vision methods use techniques such as edge detection, thresholding, and contour analysis to identify anomalies in images.

Common techniques

- Canny Edge Detection
- Sobel Operator
- Hough Transform
- Morphological Operations

Advantages

- Low computational requirements
- Easy to implement

Limitations

- Sensitive to lighting and environmental variations
- Poor performance in complex backgrounds
- Limited ability to generalize across different surfaces
- Cannot handle variations in crack shape, size, and orientation

These methods lack robustness and fail to provide reliable results in real-world scenarios.

2.4 CLOUD-BASED AI SYSTEMS

Modern systems often use deep learning models deployed on cloud servers. Images or video streams are sent to remote servers where processing is performed, and results are returned to the user.

Advantages

- High computational power using GPUs
- Capable of handling complex deep learning models
- Scalable for large datasets

Limitations

- Requires constant internet connectivity
- High latency due to data transmission

- Privacy and security concerns
- Not suitable for real-time or remote deployments

Cloud-based systems are not practical in environments where network reliability is limited.

2.5 HIGH-END INDUSTRIAL INSPECTION SYSTEMS

Industrial inspection systems use specialized hardware such as robotic arms, high-resolution cameras, and laser scanners for defect detection.

Advantages

- High accuracy and precision
- Suitable for controlled industrial environments

Limitations

- Very expensive and not cost-effective
- Requires specialized infrastructure and maintenance
- Limited portability
- Not suitable for field deployment

These systems are typically used in large-scale industries and are not accessible for general-purpose applications.

2.6 AI-BASED DETECTION SYSTEMS

Deep learning-based approaches, particularly Convolutional Neural Networks (CNNs), have significantly improved anomaly detection performance. Models such as YOLO (You Only Look Once) enable real-time object detection.

Advantages

- High accuracy and adaptability
- Capable of detecting complex patterns
- Real-time detection capability

Limitations

- Requires large annotated datasets
- High computational requirements
- Performance depends on data quality and diversity

- May produce false positives without proper training

While AI-based systems are powerful, their deployment on low-resource devices remains a challenge.

2.7 SUMMARY OF LIMITATIONS

A comparison of existing systems highlights several key challenges:

- Lack of real-time performance in many systems
- Dependency on internet or cloud infrastructure
- High cost of industrial solutions
- Poor generalization in traditional methods
- Limited portability and scalability

These limitations create a need for a system that is:

- Real-time
- Cost-effective
- Independent of internet connectivity
- Deployable on edge devices
- Accurate and robust under varying conditions

2.8 NEED FOR PROPOSED SYSTEM

To overcome the limitations of existing approaches, there is a need for an intelligent system that combines the advantages of deep learning with the efficiency of edge computing.

The proposed Edge-AI framework addresses these challenges by enabling real-time anomaly detection using a lightweight model deployed in a distributed architecture. It eliminates reliance on cloud infrastructure while maintaining high accuracy and performance, making it suitable for practical deployment in diverse environments.

System Type	Advantages	Limitations
Manual Inspection	Simple, no technology required	Time-consuming, subjective, error-prone
Classical Image Processing	Low computational cost	Sensitive to lighting, poor generalization
Cloud-Based AI Systems	High computational power	Requires internet, high latency
Industrial Systems	High accuracy	Expensive, not portable
AI-Based Systems	High accuracy, adaptable	High computation, dataset dependent

Table 2.1: Comparison of Existing Surface Anomaly Detection Systems

CHAPTER 3
PROPOSED SYSTEM

3.1 INTRODUCTION

To overcome the limitations of existing surface inspection systems, this project proposes an advanced **Edge-AI-based framework for real-time surface anomaly detection**. The system integrates computer vision, deep learning, and edge computing to provide an efficient, scalable, and portable solution.

Unlike traditional systems that rely heavily on manual inspection or cloud-based processing, the proposed approach focuses on **real-time detection with minimal latency**, while maintaining independence from continuous internet connectivity.

3.2 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system is designed to detect surface anomalies such as cracks using a lightweight deep learning model. It consists of a camera module for capturing real-time video, an edge device for handling input and streaming, and a processing unit for performing inference.

A key feature of this system is its **distributed architecture**, where tasks are divided between devices to optimize performance. The Raspberry Pi 4 is responsible for capturing and streaming video, while the computationally intensive detection is performed on a more powerful system.

This approach significantly improves system efficiency, reduces computational load on the edge device, and ensures real-time performance.

3.3 KEY FEATURES OF THE PROPOSED SYSTEM

3.3.1 Edge-Based Processing

The system leverages edge computing principles by performing data acquisition and initial processing close to the source. This reduces latency and enables faster response times compared to cloud-based systems.

3.3.2 Distributed Architecture (Major Design Improvement)

Instead of executing all operations on a single device, the system adopts a distributed design:

- **Raspberry Pi 4**: Captures video frames and streams them over a local network
- **PC/System**: Performs deep learning inference using the trained model

Advantages:

- Reduces computational burden on the Raspberry Pi
- Improves frame rate and system responsiveness
- Enables use of more optimized models

3.3.3 Real-Time Detection

The system processes video frames in real time using the YOLOv8n object detection model. It achieves an approximate performance of **15–20 frames per second (FPS)**, making it suitable for live monitoring applications.

3.3.4 Wireless Streaming

A Flask-based MJPEG streaming system is implemented to transmit video frames over a local Wi-Fi network. This allows the user to monitor detection results in real time through a web browser without requiring internet access.

3.3.5 Lightweight Deep Learning Model

The system uses **YOLOv8n**, a lightweight variant of the YOLO architecture, which provides a balance between speed and accuracy. The model is trained on a combination of publicly available and custom-annotated datasets.

3.3.6 Portability and Cost-Effectiveness

The system is designed to be portable using a Raspberry Pi, Pi camera module, and a power bank. This makes it suitable for field deployment without requiring expensive infrastructure.

3.4 DATASET PREPARATION AND ENHANCEMENT

The performance of the system is significantly improved through careful dataset preparation and refinement.

3.4.1 Dataset Composition

- Combination of publicly available annotated images
- Custom-annotated images using Roboflow
- Inclusion of negative samples (images without cracks)

3.4.2 Dataset Cleaning

- Removal of corrupted or invalid images
- Standardization of image formats and dimensions

3.4.3 Annotation Verification

- Ensured correct YOLO bounding box format
- Removed invalid or misaligned annotations

3.4.4 Impact of Dataset Improvements

- Reduced false positives
- Improved detection reliability
- Enhanced model generalization

3.5 TRAINING OPTIMIZATION

Several training strategies were applied to improve model performance:

- Training performed on Google Colab using GPU acceleration
- Image resolution reduced from 640 to 416 for speed optimization
- Batch size set to 8 and trained for multiple epochs
- Data augmentation techniques applied, including:
    - Rotation and scaling
    - Translation and flipping
    - HSV color adjustments
    - Mosaic and MixUp augmentation

These techniques improved the model's robustness under varying lighting conditions and viewing angles.

3.6 MODEL OPTIMIZATION AND PERFORMANCE ENHANCEMENT

To achieve real-time performance and high accuracy, the following optimizations were implemented:

3.6.1 Inference Optimization

- Reduced input resolution for faster processing
- Applied confidence threshold tuning (0.5–0.6)
- Implemented frame skipping to reduce latency

3.6.2 Model Conversion

- Converted trained model to ONNX format
- Enabled faster inference and better hardware compatibility

3.7 HARDWARE OPTIMIZATION

The system hardware was optimized for stable and efficient performance:

- Replaced USB webcam with Pi camera module for lower latency
- Implemented cooling mechanisms (heat sink and fan)
- Used a power bank for portable operation

These improvements ensured reliable long-duration operation in real-world conditions.

3.8 SYSTEM ARCHITECTURE

The proposed system follows a modular architecture consisting of the following stages:

1. **Image Acquisition** – Capturing video frames using the camera
2. **Preprocessing** – Resizing and formatting frames
3. **Detection** – Applying YOLOv8n model for anomaly detection
4. **Decision Logic** – Filtering results based on confidence thresholds
5. **Annotation** – Drawing bounding boxes on detected anomalies
6. **Streaming** – Transmitting output via MJPEG over local network

This modular design ensures flexibility, scalability, and ease of integration with future enhancements.

3.9 PERFORMANCE EVALUATION

After implementing all optimizations, the system achieved the following performance metrics:

- Precision ≈ 0.96
- Recall ≈ 0.91
- mAP50 ≈ 0.97
- mAP50–95 ≈ 0.76
- Real-time performance ≈ 15–20 FPS

These results indicate that the system is highly accurate, efficient, and suitable for real world deployment

Feature	Existing Systems	Proposed System
Processing	Cloud / Local heavy systems	Distributed (Edge + PC)
Cost	High	Low
Portability	Limited	High
Real-Time Performance	Moderate	High (15–20 FPS)
Internet Dependency	Required	Not required
Accuracy	Moderate–High	High (mAP50 ≈ 0.97)

Table 3.1: Comparison of Existing Systems and Proposed Edge-AI System

3.10 SUMMARY

The proposed system successfully integrates edge computing and deep learning to create a real-time anomaly detection framework. Through dataset refinement, model optimization, and hardware improvements, the system achieves high accuracy and performance.

The distributed architecture, wireless capability, and portability make it a practical and scalable solution for automated surface inspection.

CHAPTER 4
LITERATURE REVIEW

4.1 INTRODUCTION

Surface anomaly detection has evolved significantly with advancements in computer vision and deep learning. This chapter reviews key approaches and research developments relevant to automated defect detection.

4.2 TRADITIONAL METHODS

Early approaches relied on classical image processing techniques such as edge detection, thresholding, and contour analysis. While computationally efficient, these methods were highly sensitive to noise, lighting variations, and background complexity, limiting their effectiveness in real-world scenarios.

4.3 DEEP LEARNING-BASED APPROACHES

Recent advancements have introduced deep learning models such as Convolutional Neural Networks (CNNs) for image classification and object detection.

- Models like **YOLO (You Only Look Once)** enable real-time detection
- CNN-based approaches improve feature extraction and accuracy
- Transfer learning reduces training time and improves performance

However, these methods often require high computational resources and large datasets.

4.4 EDGE AI IN COMPUTER VISION

Edge AI has emerged as a solution to overcome latency and dependency on cloud systems.

- Enables processing closer to the data source
- Reduces network dependency
- Improves real-time performance

Lightweight models such as YOLOv8n make it possible to deploy AI on resource-constrained devices.

4.5 RESEARCH GAP

Despite advancements, existing systems face key limitations:

- Dependence on cloud infrastructure
- High computational cost
- Limited portability
- Poor performance in real-time edge environments

4.6 CONTRIBUTION OF PROPOSED WORK

The proposed system addresses these gaps by:

- Implementing a lightweight detection model
- Using a distributed Edge-AI architecture
- Achieving real-time performance with minimal latency
- Enhancing dataset quality for improved accuracy

CHAPTER 5

METHODOLOGY

5.1 INTRODUCTION

The methodology focuses on the algorithmic and learning approach used for anomaly detection. It describes the internal working of the deep learning model, data processing pipeline, and training strategy used to achieve high detection accuracy.

5.2 LEARNING APPROACH

The system uses a supervised learning approach, where the model is trained on labelled images containing cracks and non-crack regions.

- Input: Image
- Output: Bounding box + class label

5.3 DATA REPRESENTATION

Each image is represented as:

- Pixel matrix (H × W × C)
- Bounding box coordinates:
    - (x, y, width, height)

Annotations are stored in YOLO format.

5.4 CONVOLUTIONAL NEURAL NETWORK (CNN) CONCEPT

YOLOv8 is based on CNN.

5.4.1 Convolution Operation

Feature extraction is performed using:

$$Feature = (Input * Kernel) + Bias$$

- Detects edges, textures, patterns

5.4.2 Activation Function (ReLU)

$$f(x) = max(0, x)$$

- Introduces non-linearity

5.4.3 Pooling Layer

- Reduces feature map size

- Improves computational efficiency

5.5 YOLOV8 DETECTION MECHANISM

Unlike traditional CNNs, YOLO performs:

- Single-stage detection
- Processes image in one pass

Steps:

1. Image divided into grids
2. Each grid predicts bounding boxes
3. Confidence score assigned
4. Non-Max Suppression applied

5.6 LOSS FUNCTION

YOLO optimizes multiple losses:

- Bounding box loss
- Classification loss
- Confidence loss

$$TotalLoss = Localization + Classification + Confidence$$

5.7 TRAINING STRATEGY

The model is trained using:

- Transfer learning (pretrained weights)
- Backpropagation
- Gradient descent optimization

Training Process

1. Forward pass
2. Loss calculation
3. Backpropagation
4. Weight update

5.8 EVALUATION METRICS

- Precision

- Recall
- F1 Score
- mAP

These metrics measure detection accuracy and reliability.

5.9 INFERENCE LOGIC

During testing:

- Model predicts bounding boxes
- Applies threshold filtering
- Removes duplicates using NMS

5.10 SUMMARY

This methodology focuses on:

- CNN-based learning
- YOLO detection logic
- Training and evaluation process

It ensures accurate and efficient anomaly detection.

CHAPTER 6

HARDWARE AND SOFTWARE REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

- Raspberry Pi 4 (Edge Device)
- Pi Camera Module
- Laptop / PC (for inference)
- Power Bank (for portability)
- Cooling System (heat sink and fan)

6.2 SOFTWARE REQUIREMENTS

- Operating System: Raspberry Pi OS / Windows
- Programming Language: Python
- Libraries and Frameworks:
    - OpenCV
    - Ultralytics YOLOv8
    - Flask
    - ONNX Runtime
    - NumPy

6.3 FUNCTIONAL REQUIREMENTS

- Capture real-time video from camera
- Perform anomaly detection using deep learning
- Display annotated output
- Stream video over local network
- Operate without internet connectivity

6.4 NON-FUNCTIONAL REQUIREMENTS

- Real-time performance (15–20 FPS)
- High accuracy and reliability
- Low latency
- Portability
- Scalability for future enhancements

CHAPTER 7
SYSTEM ARCHITECURE

7.1 INTRODUCTION

System architecture defines the structure and workflow of the proposed Edge-AI framework for real-time surface anomaly detection. It illustrates how different components interact to capture, process, detect, and visualize anomalies in real time.

The architecture is designed to be modular, efficient, and scalable, ensuring smooth data flow and optimal performance across all stages of the system.

7.2 OVERVIEW OF THE ARCHITECTURE

The proposed system follows a **distributed Edge-AI architecture**, where tasks are divided between multiple components to improve efficiency and performance.
The architecture consists of three main layers:

- **Input Layer** – Captures real-time video data
- **Processing Layer** – Performs preprocessing and anomaly detection
- **Output Layer** – Displays annotated results and streams video

A key feature of this architecture is the separation of responsibilities between the Raspberry Pi and the processing system, enabling real-time performance without overloading the edge device.

7.3 SYSTEM COMPONENTS

The system consists of the following major components:

7.3.1 Camera Module (Input Acquisition)

- Captures real-time video frames
- Uses Pi Camera Module for low-latency input
- Supports continuous frame acquisition

Role:

Acts as the primary data source for the system.

7.3.2 Edge Device – Raspberry Pi 4

- Controls the camera module
- Handles frame capture and initial processing
- Streams video over local Wi-Fi network

Role:

Serves as the edge device responsible for data acquisition and transmission.

7.3.3 Processing System (PC / Laptop)

- Receives video stream from Raspberry Pi
- Performs deep learning inference using YOLOv8n model
- Applies detection and decision logic

Role:

Handles computationally intensive tasks to ensure high performance.

7.3.4 Detection Model (YOLOv8n)

- Lightweight object detection model
- Detects surface anomalies such as cracks
- Outputs bounding boxes and confidence scores

Role:

Core component responsible for anomaly detection.

7.3.5 Streaming Module (Flask-Based MJPEG Streaming)

- Transmits video frames over local network
- Uses HTTP-based MJPEG streaming
- Accessible through a web browser

Role:

Enables real-time monitoring without requiring internet connectivity.

7.3.6 Display and Visualization Module

- Displays annotated video frames
- Shows bounding boxes and detection results
- Includes FPS counter for performance monitoring

Role:

Provides user-friendly visualization of detection results.

7.4 WORKING OF THE SYSTEM

The system operates through a sequence of steps that ensure real-time detection and visualization:

1. **Frame Capture**

   The camera module captures continuous video frames from the surface.

2. **Frame Transmission**

   The Raspberry Pi streams the captured frames over a local Wi-Fi network using MJPEG streaming.

3. **Frame Reception**

   The processing system receives the video stream.

4. **Preprocessing**

   Frames are resized and normalized to match the model input requirements.

5. **Anomaly Detection**

   The YOLOv8n model processes each frame and detects anomalies by generating bounding boxes and confidence scores.

6. **Decision Logic**

   Predictions are filtered based on confidence thresholds to ensure accurate detection.

7. **Annotation**

   Detected anomalies are highlighted with bounding boxes on the video frames.

8. **Display and Streaming**

   The annotated video is displayed and streamed in real time for monitoring.

7.5 DISTRIBUTED ARCHITECTURE DESIGN

A major improvement in the system is the implementation of a **distributed architecture**.

Traditional Approach

- All processing performed on Raspberry Pi
- Leads to high latency and low FPS

Proposed Approach

- Raspberry Pi → Frame capture and streaming
- PC → Model inference and detection

Benefits

- Reduced computational load on Raspberry Pi
- Improved frame rate (15–20 FPS)
- Enhanced system scalability
- Better utilization of available resources

This design ensures efficient real-time performance while maintaining portability.

7.6 DATA FLOW IN THE SYSTEM

The data flow within the system can be summarized as follows:

- Input video is captured by the camera
- Frames are streamed via Raspberry Pi
- Processing system receives and processes frames
- YOLOv8n model detects anomalies
- Results are annotated and displayed
- Output is streamed for real-time monitoring

This structured flow ensures smooth and efficient operation of the system.

7.7 ADVANTAGES OF THE ARCHITECTURE

The proposed architecture offers several advantages:

- **Real-Time Processing** – Achieves low-latency detection
- **Scalability** – Can be extended to other anomaly types
- **Portability** – Suitable for field deployment
- **Cost-Effective** – Uses affordable hardware components
- **Wireless Operation** – Eliminates need for internet connectivity
- **Flexibility** – Modular design allows easy upgrades

Figure 7.1: System architecture

7.8 SUMMARY

The system architecture is designed to efficiently integrate edge computing and deep learning for real-time anomaly detection. By distributing tasks between the Raspberry Pi and the processing system, the architecture achieves high performance while maintaining portability and scalability.

This structured design forms the backbone of the proposed system and enables reliable operation in real-world environments.

CHAPTER 8
SYSTEM MODULES

8.1 INTRODUCTION

The system is designed using a modular architecture where each module performs a specific function. This improves scalability, maintainability, and efficiency.

8.2 MODULES OVERVIEW

The system consists of the following modules:

- Image Acquisition Module
- Preprocessing Module
- Detection Module
- Decision Module
- Streaming Module

8.3 IMAGE ACQUISITION MODULE

Captures real-time video using the Pi camera module.

Functions:

- Continuous frame capture
- Resolution control
- Frame consistency

8.4 PREPROCESSING MODULE

Prepares frames for model input.

Functions:

- Resizing (416 × 416)
- Normalization
- Format conversion

8.5 DETECTION MODULE

Core module using YOLOv8n.

Functions:

- Detect anomalies

- Generate bounding boxes
- Output confidence scores

8.6 DECISION MODULE

Filters predictions.

Functions:

- Apply confidence threshold (0.5–0.6)
- Reduce false positives

8.7 STREAMING MODULE

Handles wireless output.

Functions:

- MJPEG streaming via Flask
- Browser-based viewing

8.8 SUMMARY

Each module works independently yet collaboratively, ensuring efficient real-time detection.

CHAPTER 9

DATA FLOW DIAGRAM (DFD)

9.1 INTRODUCTION

DFD represents how data flows through the system from input to output.

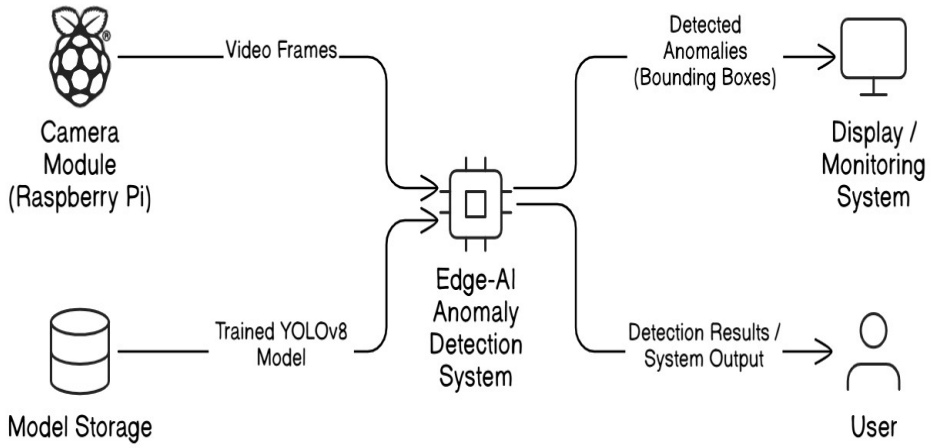

Figure 9.1: Level 0 Data Flow Diagram (Context Diagram)

9.2 LEVEL 0 (CONTEXT DIAGRAM)

Entities:

- User
- System
- Output

Flow:

User → System → Output

9.3 LEVEL 1 DFD

Breakdown:

- Input → Camera
- Preprocessing → Frame cleaning
- Detection → YOLO model
- Output → Display

9.4 LEVEL 2 DFD

Detailed flow:

- Frame capture
- Preprocessing
- Feature extraction
- Detection
- Decision
- Output

CHAPTER 10
UML DIAGRAMS

10.1 INTRODUCTION

Unified Modeling Language (UML) diagrams are used to represent the structure and behavior of the proposed Edge-AI Anomaly Detection System. These diagrams provide a clear understanding of how the system operates, how components interact, and how users engage with the system.

The UML diagrams included in this chapter are:

- Use Case Diagram
- Sequence Diagram
- Class Diagram

10.2 USE CASE DIAGRAM

The Use Case Diagram represents the interaction between the user and the system. It highlights the primary functionalities available to the user.

In this system, the user primarily interacts through passive observation and monitoring.

Key Use Cases:

- View Live Video Stream
- Monitor Detection Results
- Observe System Performance

The user does not directly control the system but observes real-time outputs generated by the detection process.

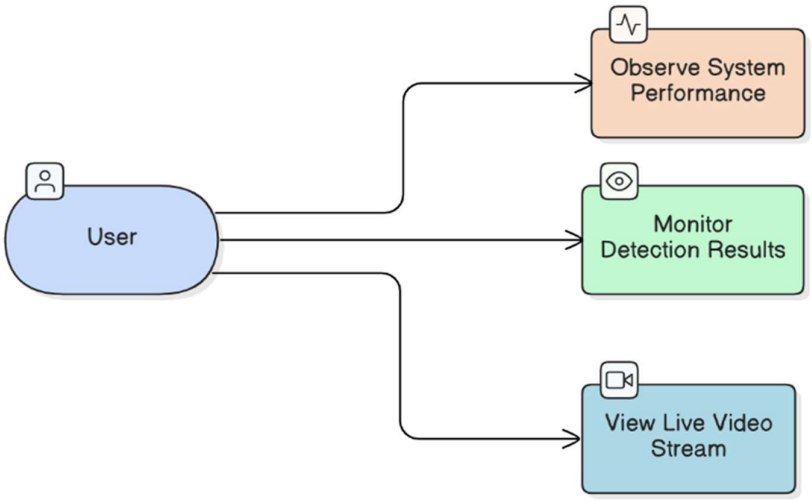

Fig 10.1: Use Case Diagram of the Proposed System

10.3 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the step-by-step flow of operations in the system. It shows how data moves through different stages from input to output.

Sequence Flow:

1. Frames are captured and streamed from the camera
2. The processing system receives the frames
3. The detection model performs anomaly detection
4. Results are generated and displayed
5. The user views the output This diagram highlights the real-time processing capability of the system.

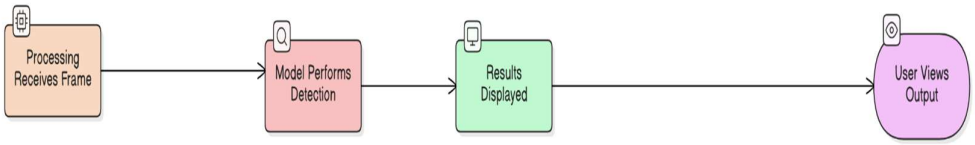

Fig 10.2: Sequence Diagram of the Proposed System

10.4 CLASS DIAGRAM

The Class Diagram represents the structural components of the system and their relationships.

Main Components:

- Camera Module
- Streaming Module
- Processing System
- Detection Model
- Output Display
- User Interface

These components work together to capture, process, detect, and display anomalies in real time.

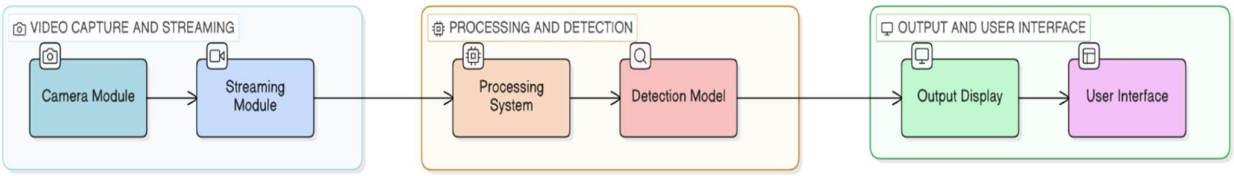

Fig 10.3: Class Diagram of the Proposed System

10.5 SUMMARY

The UML diagrams provide a comprehensive view of the system's functionality, workflow, and structure. Together, they demonstrate how the proposed system achieves real-time anomaly detection through efficient interaction between components and user monitoring.

CHAPTER 11
IMPLEMENTATION

11.1 INTRODUCTION

The implementation phase involves the development, integration, and execution of all components of the proposed Edge-AI framework for real-time surface anomaly detection. This includes dataset preparation, model training, optimization, system integration, and deployment of a real-time detection pipeline.

The primary objective of this phase is to achieve high detection accuracy while maintaining real-time performance using deep learning techniques combined with edge computing principles.

11.2 IMPLEMENTATION OVERVIEW

The implementation was carried out in a structured and systematic manner through the following stages:

1. Dataset collection and preparation
2. Model training using YOLOv8n
3. Model optimization and conversion
4. System integration with edge hardware
5. Real-time streaming and detection
6. Performance tuning and evaluation

Each stage was designed to ensure robustness, scalability, and efficiency.

11.3 DATASET COLLECTION AND PREPARATION

11.3.1 Dataset Composition

The dataset used for training consists of:

- Publicly available annotated crack images
- Custom-annotated images
- Negative samples (non-crack images)

The inclusion of negative samples significantly improved model reliability by reducing false positive detections.

11.3.2 Dataset Cleaning

To ensure data quality:

- Corrupted and invalid images were removed
- Image formats and dimensions were standardized
- Images were re-encoded to eliminate hidden inconsistencies

11.3.3 Annotation Verification

Annotations were carefully validated:

- Verified YOLO bounding box format
- Removed invalid bounding boxes
- Corrected misaligned annotations

11.3.4 Dataset Structuring

The dataset was organized into:

- Training set (train/)
- Validation set (val/)
- Testing set (test/)

This ensured proper model training and evaluation.

11.4 MODEL TRAINING

11.4.1 Training Environment

- Platform: Google Colab (GPU-enabled)
- Framework: Ultralytics YOLOv8

11.4.2 Training Configuration

- Model: YOLOv8n (lightweight variant)
- Image size: 416 × 416
- Batch size: 8
- Epochs: 50–80 (multiple training iterations)

11.4.3 Data Augmentation

To enhance generalization, the following techniques were applied:

- Rotation and scaling
- Translation and horizontal flipping

- HSV augmentation
- Mosaic and MixUp augmentation

These techniques improved robustness under varying environmental conditions.

11.4.4 Training Optimization

- Multi-scale training disabled for stability
- Hyperparameters tuned for convergence
- Image resolution reduced to improve speed

These optimizations resulted in stable training and improved model performance.

11.5 Training Parameters

The training of the YOLOv8n model requires careful selection of hyperparameters to ensure optimal performance. Parameters such as image size, batch size, number of epochs, and training platform play a crucial role in achieving a balance between detection accuracy and computational efficiency. The following configuration was adopted to optimize both model performance and real-time inference capability.

Parameter	Value
Model	YOLOv8n
Epochs	50–80
Batch	8
Image Size	416 × 416
Framework Ultralytics	YOLOv8
Training Platform	Google Colab

Table 11.1 Training configuration parameters

11.6 MODEL OPTIMIZATION

11.6.1 Inference Optimization

To achieve real-time performance:

- Reduced input resolution
- Tuned confidence threshold (0.5–0.6)
- Implemented frame skipping

11.6.2 Model Conversion

The trained model was converted to **ONNX format**.

Advantages:

- Faster inference
- Better hardware compatibility
- Flexibility for deployment

11.7 SYSTEM INTEGRATION

11.7.1 Hardware Setup

The system consists of:

- Raspberry Pi 4 (edge device)
- Pi Camera Module
- Power bank for portability
- Cooling system (heat sink and fan)

11.7.2 Software Stack

- Python
- OpenCV
- Ultralytics YOLOv8
- Flask (for streaming)
- ONNX Runtime

11.7.3 Distributed Processing Implementation

The system uses a distributed architecture:

- Raspberry Pi → Captures and streams video
- PC → Performs model inference

This approach significantly reduces computational load on the Raspberry Pi and improves performance.

11.8 REAL-TIME DETECTION SYSTEM

The real-time detection pipeline operates as follows:

1. Camera captures video frames
2. Raspberry Pi streams frames over local Wi-Fi
3. PC receives video stream
4. Frames are preprocessed
5. YOLOv8n model performs detection

6. Bounding boxes are generated
7. Results are displayed and streamed

11.9 STREAMING IMPLEMENTATION

- Implemented using **Flask-based MJPEG streaming**
- Frames transmitted over local Wi-Fi network
- Accessible through a web browser

Features

- Real-time video transmission
- No internet dependency
- Compatible with mobile hotspot networks

11.10 HARDWARE OPTIMIZATION

Several hardware-level improvements were implemented:

- Replaced USB webcam with Pi camera module for lower latency
- Added cooling system for thermal stability
- Enabled portable operation using power bank

These improvements ensured reliable performance during long-duration operation.

11.11 VISUAL AND INTERFACE IMPROVEMENTS

To enhance usability and monitoring:

- Increased display resolution (1280 × 720)
- Added FPS counter
- Cleaned output interface
- Improved bounding box visualization

11.12 CAMERA CALIBRATION AND IMAGE PROCESSING

- Fixed RGB to BGR color mismatch
- Enabled auto white balance (AWB)
- Improved image quality and color consistency

These changes improved detection accuracy under varying lighting conditions.

11.13 PERFORMANCE EVALUATION

Metric	Value
Precision	0.96
Recall	0.91
mAP50	0.97
mAP50-95	0.76
FPS	15-20

Table 11.2: Performance Metrics of YOLOv8n Model

11.14 CHALLENGES FACED

During implementation, several challenges were encountered:

- Dataset limitations and imbalance
- False positives in early stages
- Performance constraints on Raspberry Pi
- Variations in lighting and camera angles

These challenges were addressed through dataset refinement, model tuning, and system optimization.

11.15 RESULTS

- Accurate crack detection achieved
- Real-time performance maintained
- Reliable operation under different conditions

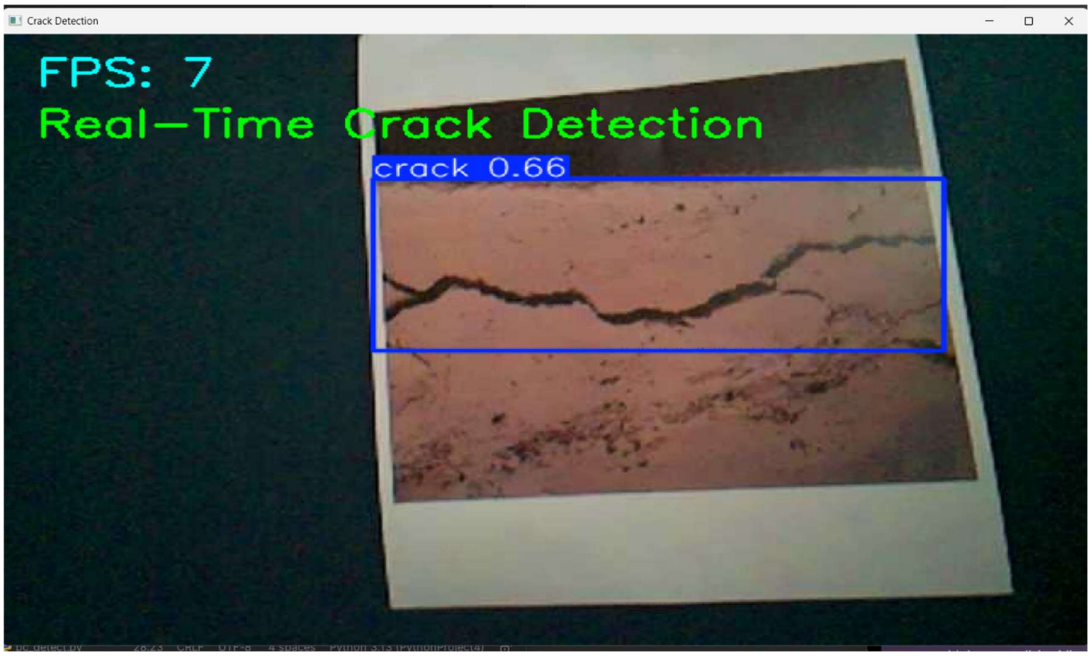

Figure 11.15.1 Sample Outputs and Results

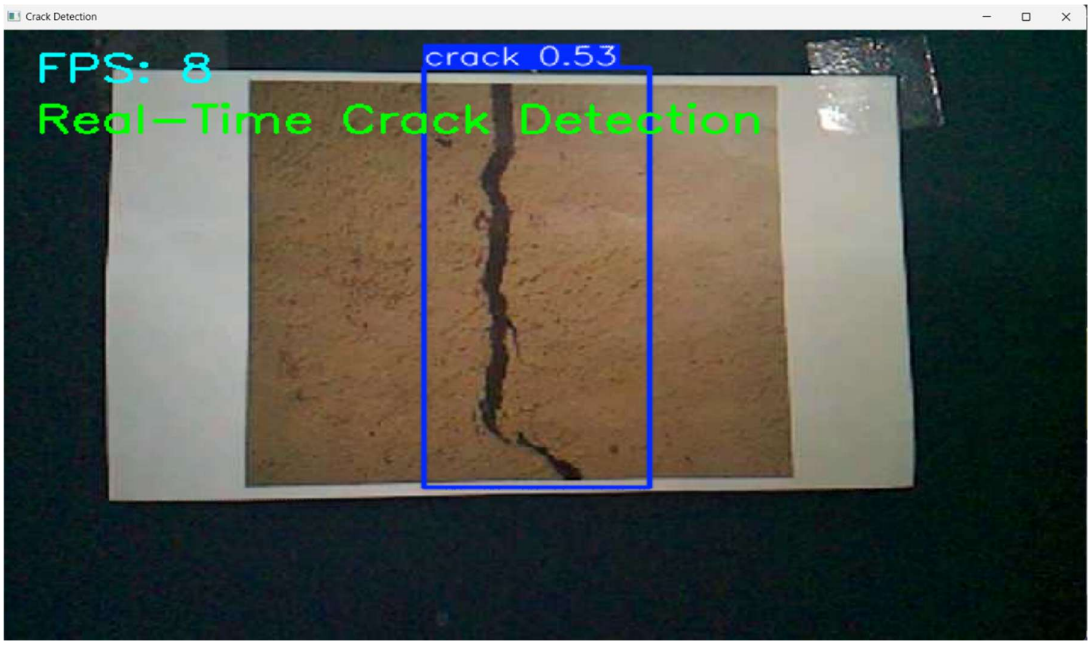

Figure 11.15.2: Input Frame Captured from Camera

11.16 SUMMARY

The implementation successfully integrates deep learning, edge computing, and real-time streaming to create an efficient anomaly detection system. Through careful optimization at the dataset, model, and system levels, the proposed solution achieves high accuracy and real-time performance.

The system demonstrates practical applicability and serves as a strong foundation for future enhancements in automated inspection systems.

CHAPTER 12
RESULTS AND DISCUSSION

12.1 INTRODUCTION

This chapter presents the experimental results obtained from the trained YOLOv8n model and analyzes its performance in detecting surface anomalies. The discussion focuses on interpreting the model outputs, evaluating detection accuracy, and assessing real-time performance using actual training and testing results.

12.2 TRAINING RESULTS ANALYSIS

The training results illustrate the behaviour of the model during the learning process across multiple epochs.

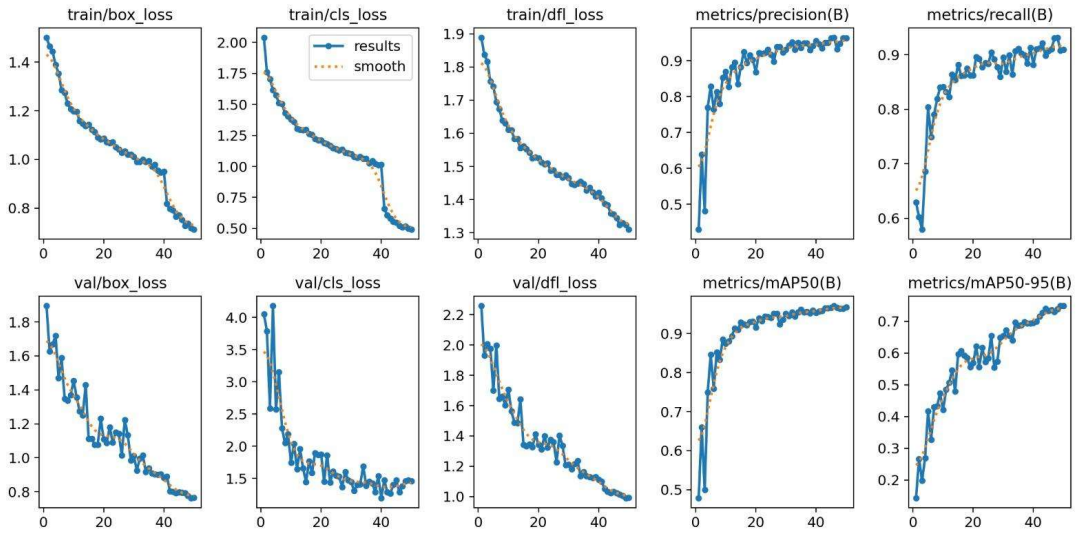

Figure 12.1: Training and Validation Metrics

Observations:

- All loss components show a consistent decreasing trend
- Validation metrics improve alongside training metrics
- No sudden fluctuations or divergence observed

Discussion:

The steady reduction in loss values indicates that the model has learned meaningful features from the dataset. The similarity between training and validation trends confirms that the model generalizes well and does not suffer from overfitting.

12.3 PRECISION–RECALL ANALYSIS

The precision–recall curve provides insight into the model's ability to balance correct detections and missed detections across different confidence thresholds.

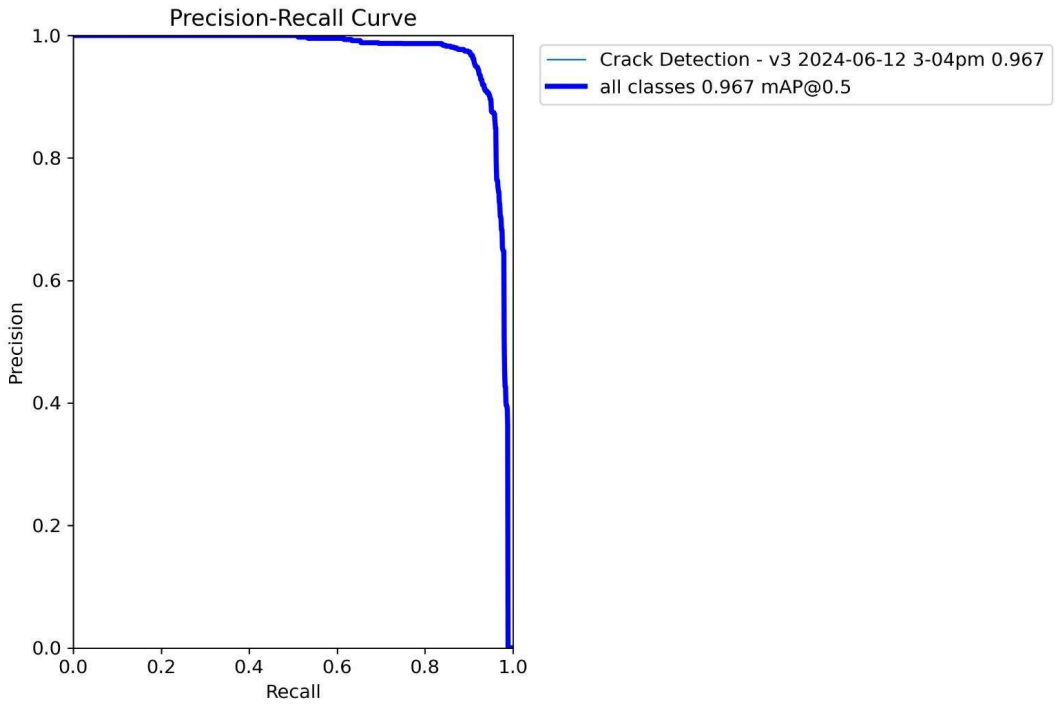

Figure 12.2: Precision–Recall Curve (BoxPR_curve.png)

Observations:

- Curve remains close to the upper boundary
- High area under the curve

Discussion:

A high precision–recall curve indicates that the model can correctly identify anomalies while maintaining a low rate of false detections. This confirms strong detection capability across different thresholds.

12.4 PRECISION ANALYSIS

The precision curve illustrates how the accuracy of predictions varies with changes in the confidence threshold.

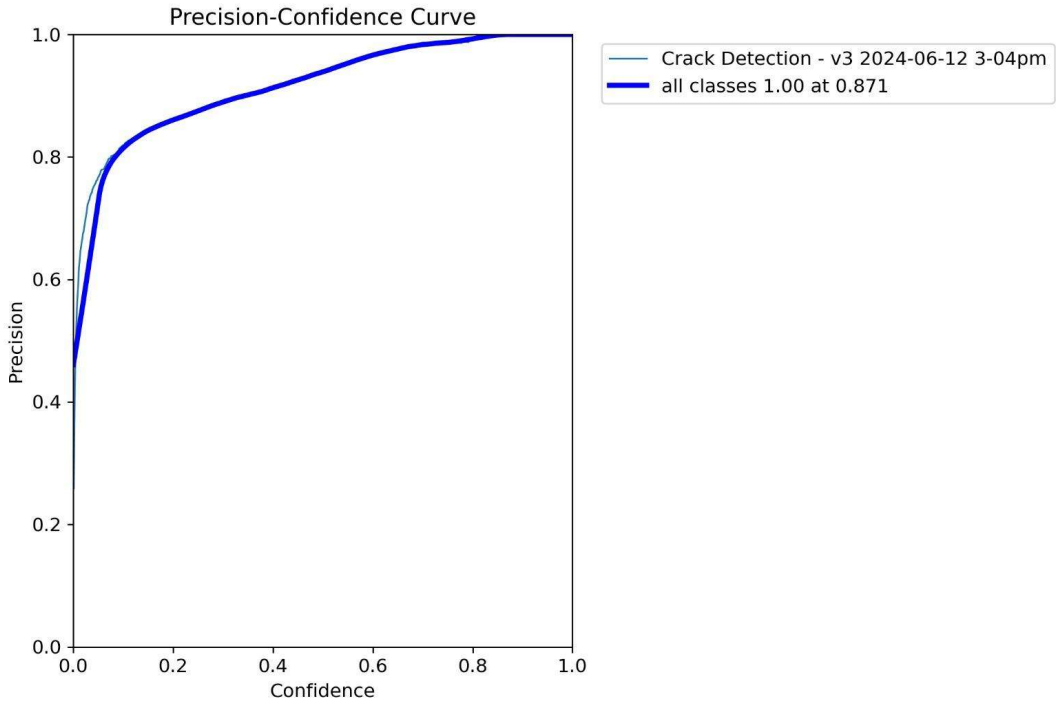

Figure 12.3: Precision Curve

Observations:

- Precision increases with higher confidence levels
- Stabilizes near maximum value

Discussion:

This behaviour shows that predictions become more reliable at higher confidence thresholds. It helps in selecting an appropriate threshold to minimize incorrect detections.

12.5 RECALL ANALYSIS

The recall curve represents the model's ability to detect all relevant anomalies as the confidence threshold changes.

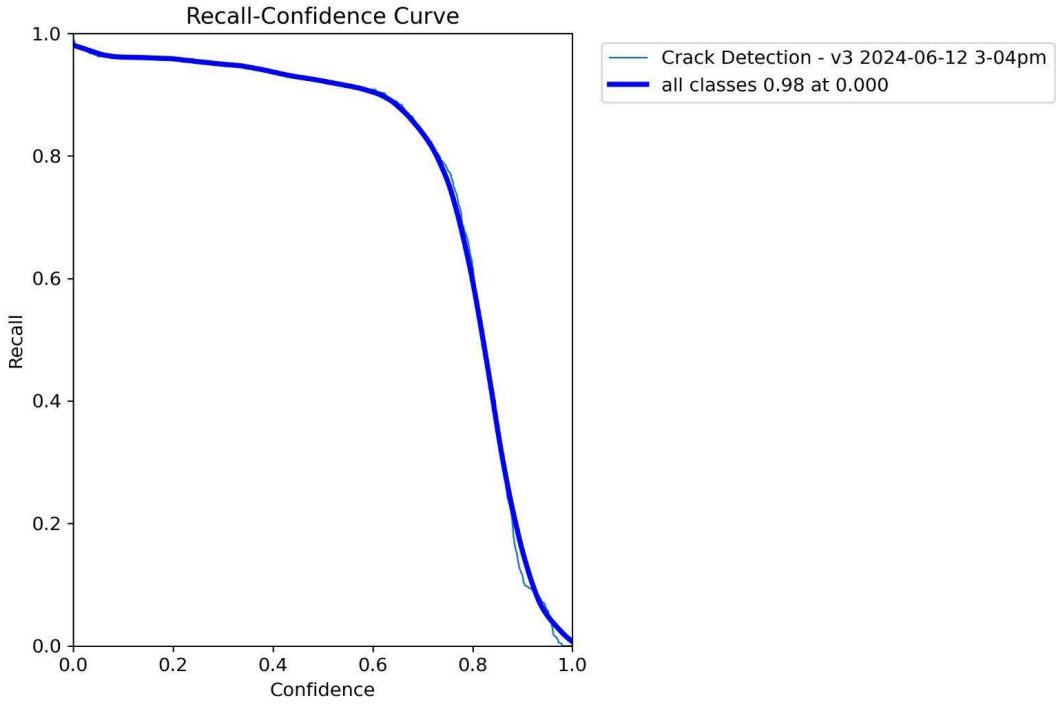

Figure 12.4: Recall Curve

Observations:

Recall decreases gradually as confidence increases

Discussion:

This reflects the typical trade-off between detecting all anomalies and maintaining prediction confidence. Lower thresholds increase detection coverage, while higher thresholds improve precision.

12.6 F1 SCORE ANALYSIS

The F1 score curve combines precision and recall into a single metric, helping identify the optimal operating point of the model.

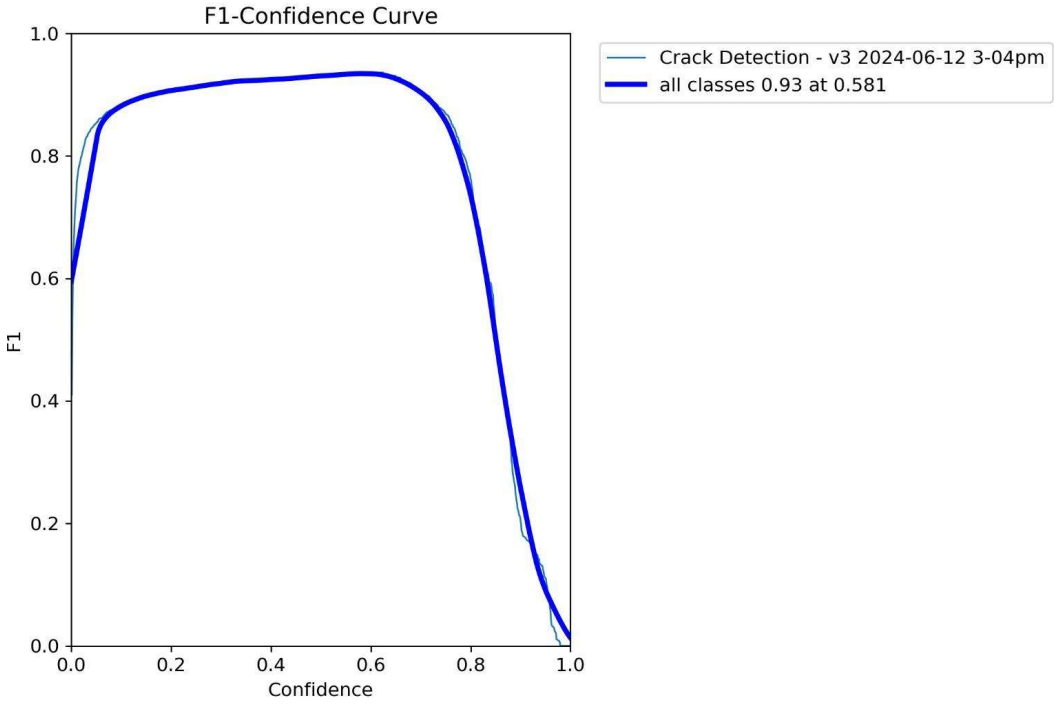

Figure 12.5: F1 Score Curve

Observations:

- Peak F1 score ≈ 0.93
- Optimal confidence ≈ 0.55–0.60

Discussion:

The F1 score represents the balance between precision and recall. The peak value indicates the most effective operating point for the model during inference.

12.7 CONFUSION MATRIX ANALYSIS

The confusion matrix provides a detailed breakdown of correct and incorrect predictions made by the model.

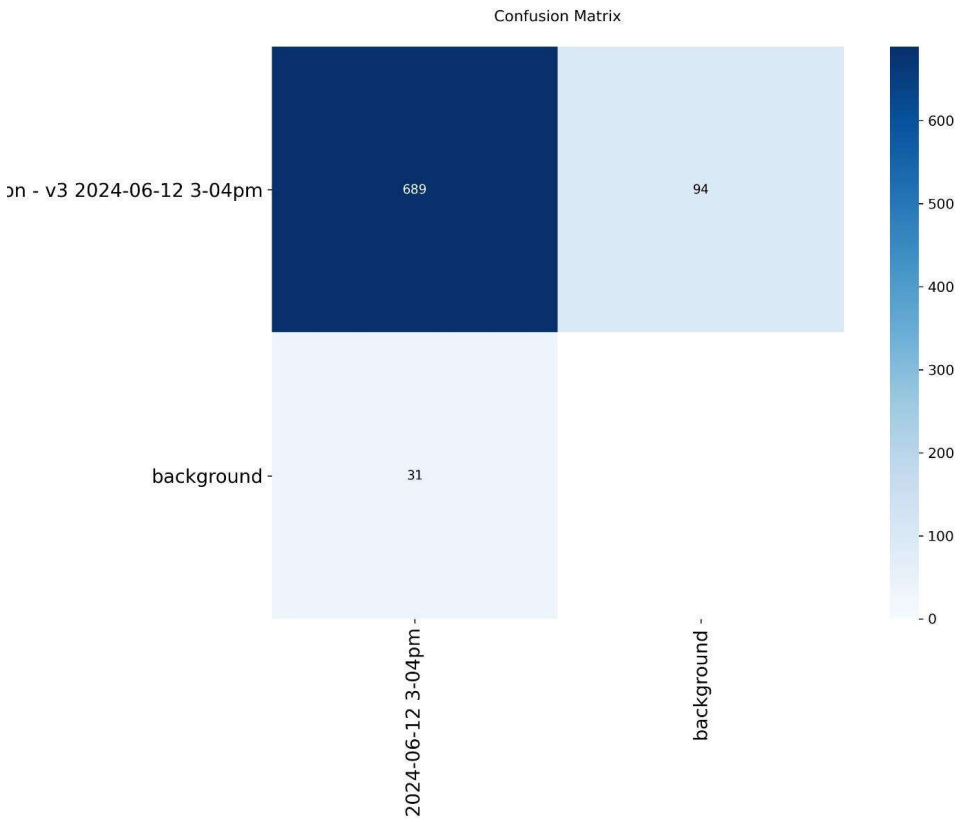

Figure 12.6: Confusion Matrix

Observations:

- High number of correct detections
- Low missed detections
- Some background misclassification

Discussion:

The confusion matrix shows that the model is highly effective in distinguishing between defective and non-defective regions. Minor misclassifications occur due to complex textures and lighting variations.

12.8 REAL-TIME PERFORMANCE ANALYSIS

The system operates at approximately **15–20 frames per second**, ensuring continuous and smooth detection.

Discussion:

This performance level is sufficient for real-time monitoring applications and confirms that the system meets the requirements for edge-based deployment without significant latency.

12.9 OVERALL DISCUSSION

The experimental results demonstrate that the proposed system achieves a strong balance between accuracy and efficiency.

Key Findings:

- High detection accuracy across all metrics
- Stable model behavior during training
- Effective performance under varying conditions
- Real-time capability achieved

Limitations Observed:

- Slight misclassification in textured backgrounds
- Performance dependent on dataset diversity

12.10 SUMMARY

This chapter analysed the model performance using actual training outputs and evaluation metrics. The results confirm that the YOLOv8n model is capable of accurate and efficient surface anomaly detection in real-time environments.

CHAPTER 13
FEASIBILITY STUDY

13.1 INTRODUCTION

The feasibility study evaluates the practicality of implementing the proposed system in real-world scenarios.

13.2 TECHNICAL FEASIBILITY

The system uses widely available technologies such as Python, OpenCV, and YOLOv8, making it technically feasible. The use of a lightweight model and distributed architecture ensures compatibility with resource-constrained devices.

13.3 ECONOMIC FEASIBILITY

The system is cost-effective as it uses low-cost hardware such as Raspberry Pi and open-source software. It provides a viable alternative to expensive industrial inspection systems.

13.4 OPERATIONAL FEASIBILITY

The system is easy to deploy and operate. It requires minimal setup and does not depend on internet connectivity, making it suitable for field environments.

13.5 FEASIBILITY SUMMARY

The project is:

- Technically feasible
- Economically viable
- Operationally practical

CHAPTER 14
APPLICATIONS

14.1 INTRODUCTION

The proposed Edge-AI Anomaly Detection System has wide applicability across multiple domains where real-time monitoring and defect detection are critical. By combining deep learning with edge computing, the system enables efficient, low-latency, and cost-effective inspection solutions.

14.2 INFRASTRUCTURE MONITORING

The system can be used for detecting cracks and structural defects in infrastructure such as:

- Buildings
- Bridges
- Roads
- Tunnels

Early detection of anomalies helps in preventing structural failures and reduces maintenance costs. The real-time capability allows continuous monitoring without manual inspection.

14.3 INDUSTRIAL INSPECTION

In industrial environments, the system can be used for:

- Surface defect detection in manufactured products
- Quality control in production lines
- Inspection of machinery components

This reduces human error, improves consistency, and enhances production efficiency.

14.4 SMART CITIES AND PUBLIC SAFETY

The system can be integrated into smart city infrastructure for:

- Monitoring public structures
- Detecting damage in urban environments
- Ensuring safety in high-traffic areas

- This contributes to safer and more efficient urban management systems.

14.5 REMOTE AND PORTABLE INSPECTION

Due to its portable design using Raspberry Pi and wireless streaming:

- The system can be deployed in remote areas
- Suitable for field inspections
- Can operate using mobile hotspots and power banks

This makes it highly flexible and practical for real-world deployment.

14.6 TRANSPORTATION AND ROAD SAFETY

The system can be applied in:

- Road surface inspection
- Detection of cracks and potholes
- Maintenance planning for highways

This improves road safety and reduces long-term repair costs.

14.7 FUTURE APPLICATIONS

With further enhancements, the system can be extended to:

- Drone-based inspection systems
- Integration with IoT platforms
- Automated alert and reporting systems
- Multi-anomaly detection (corrosion, dents, etc.)

These advancements can significantly expand the scope of the system.

14.8 SUMMARY

The proposed system demonstrates strong potential across various domains by providing a scalable, real-time, and cost-effective solution for anomaly detection. Its flexibility, portability, and accuracy make it suitable for both industrial and real-world applications.

CHAPTER 15
CHALLENGES AND FUTURE WORK

15.1 CHALLENGES FACED

15.1.1 Dataset Challenges

- Limited availability of high-quality crack datasets
- Need for manual annotation
- Dataset imbalance

15.1.2 Model Training Challenges

- Overfitting during early stages
- Need for hyperparameter tuning
- Balancing accuracy and speed

15.1.3 Real-Time Performance Challenges

- Limited processing power of Raspberry Pi
- Latency issues during inference
- Need for optimization techniques

15.1.4 Environmental Challenges

- Lighting variations
- Different surface textures
- Camera angle variations

15.2 LIMITATIONS

- Performance depends on dataset quality
- Limited to specific types of anomalies
- Requires proper camera positioning
- May struggle in extreme lighting conditions

15.3 FUTURE IMPROVEMENTS

- Integration of more advanced models
- Support for multiple anomaly types

- Mobile application integration
- Fully edge-based optimized inference

CHAPTER 16
CONCLUSION

16.1 SUMMARY OF THE PROJECT

This project successfully developed an Edge-AI-based framework for real-time surface anomaly detection using deep learning techniques. The system integrates computer vision, distributed processing, and wireless streaming to provide an efficient and scalable solution.

16.2 KEY ACHIEVEMENTS

- Real-time anomaly detection using YOLOv8n
- High accuracy with optimized dataset and training
- Distributed architecture for improved performance
- Wireless and portable system design
- Successful deployment using Raspberry Pi

16.3 SIGNIFICANCE OF THE WORK

The system demonstrates the practical application of Edge AI in real-world inspection tasks. It provides a cost-effective alternative to traditional inspection methods and reduces reliance on cloud infrastructure.

16.4 FUTURE SCOPE

- Expansion to detect multiple defect types
- Integration with IoT systems
- Deployment on fully optimized edge hardware
- Use of advanced deep learning architectures

16.5 FINAL REMARKS

The project highlights the potential of combining edge computing and deep learning to create intelligent, real-time systems. With further enhancements, the proposed framework can be extended to various industrial and infrastructure monitoring applications.

REFERENCES

Books

1. Ian Goodfellow, Yoshua Bengio, and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Richard Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010.
3. Simon J. D. Prince, *Computer Vision: Models, Learning, and Inference*, Cambridge University Press, 2012.

Research Papers

4. Joseph Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
5. Alexey Bochkovskiy et al., "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020.
6. Ultralytics, "YOLOv8: Next Generation Object Detection Model," 2023.

Web Resources

7. Ultralytics YOLOv8 Documentation: https://docs.ultralytics.com
8. OpenCV Documentation: https://docs.opencv.org
9. Raspberry Pi Official Documentation: https://www.raspberrypi.org/documentation
10. Flask Documentation: https://flask.palletsprojects.com
11. ONNX Runtime Documentation: https://onnxruntime.ai

Tools and Platforms

12. Google Colab: https://colab.research.google.com
13. Roboflow (Dataset Annotation Tool): https://roboflow.com

APPENDICES

```
[ ]
!unzip /content/drive/MyDrive/dataset_new_2.zip -d /content/
>
Show hidden output
```

Figure A.1 Dataset extraction

```
[ ]
!pip install ultralytics
>
Show hidden output
```

Figure A.2 Library Installation

```
[ ]
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="/content/dataset_new_2/data.yaml",
    epochs=50,
    imgsz=640,
    batch=8,

    multi_scale=False,
    patience=10,

    hsv_h=0.015,
    hsv_s=0.7,
    hsv_v=0.4,
    degrees=10,
    translate=0.1,
    scale=0.5,
    fliplr=0.5,
    mosaic=1.0,
    mixup=0.2
)
```

Figure A.3 Model Training Code

```python
import cv2
import time
from ultralytics import YOLO

# Load your trained model
model = YOLO("best9.pt")   # or best9.onnx if using ONNX

# Pi stream URL
PI_STREAM = "http://172.20.10.11:5000"

cap = cv2.VideoCapture(PI_STREAM)

# Reduce buffering (important for latency)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)

#  Create large window
cv2.namedWindow("Crack Detection", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Crack Detection", 1280, 720)

prev_time = time.time()

while True:
    ret, frame = cap.read()
    if not ret:
        continue

    #  Skip old frames to reduce lag
    for _ in range(2):
        cap.read()

    # YOLO inference
    results = model(frame, imgsz=416, conf=0.5, verbose=False)
```

```python
    # YOLO inference
    results = model(frame, imgsz=416, conf=0.5, verbose=False)

    # Draw results
    frame = results[0].plot()

    #FPS counter
    curr_time = time.time()
    fps = 1 / (curr_time - prev_time)
    prev_time = curr_time

    # Display FPS
    cv2.putText(frame, f"FPS: {int(fps)}", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 0), 2)

    # Display title
    cv2.putText(frame, "Real-Time Crack Detection", (20, 80),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

    # Show output
    cv2.imshow("Crack Detection", frame)

    if cv2.waitKey(1) == 27:  # ESC key
        break

cap.release()
cv2.destroyAllWindows()
```

Figure A.4 To run inference on pc

```python
@app.route('/')
def video_feed():
    return Response(
        generate(),
        mimetype='multipart/x-mixed-replace; boundary=frame'
    )


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, threaded=True)
```

```python
import cv2
from flask import Flask, Response

app = Flask(__name__)

# 🔥 Use Pi Camera (libcamera backend)
cap = cv2.VideoCapture(0)

# 🔥 Lower resolution = faster stream
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

def generate():
    while True:
        ret, frame = cap.read()
        if not ret:
            continue

        # 🔥 Encode frame (lower quality = faster)
        _, buffer = cv2.imencode(
            '.jpg',
            frame,
            [int(cv2.IMWRITE_JPEG_QUALITY), 50]
        )

        frame_bytes = buffer.tobytes()

        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')
```

Figure A.5: Streaming from edge device